

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5

9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I G Vyshnavi (192525260) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:vyshnavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Pseudocode for the complete query processing pipeline

Input: SQL query

Output: Query result

ALGORITHM Query_Processing_Pipeline(SQL_Query)

BEGIN

 INPUT SQL_Query

 // Step 1: Parsing

 Parse SQL_Query

 Check syntax and semantic correctness

 IF Query is invalid THEN

 Display "Invalid SQL Query"

 STOP

 END IF

 // Step 2: Translation

 Convert SQL_Query into an internal query representation

 Construct initial query tree

 // Step 3: Optimization

 Generate possible execution plans

 Estimate cost of each plan

 Select the plan with minimum estimated cost

 // Step 4: Execution

 Send selected plan to query execution engine

 Execute relational operations

 // Step 5: Result generation

 Collect output tuples

 Display query result

END

Flow:

SQL Input → Parsing → Translation → Optimization → Execution → Result Output

Q2. Pseudocode for cost-based query optimization using different join orderings

Input: Relations, join conditions, relation statistics

Output: Minimum-cost join ordering

ALGORITHM Cost_Based_Optimization(Relations)

BEGIN

 Generate all possible join orderings

 bestPlan ← NULL

 minimumCost ← INFINITY

 FOR each JoinOrder in allJoinOrders DO

 currentCost ← 0

 intermediateSize ← 0

 FOR each Join in JoinOrder DO

 Estimate number of tuples

 Estimate number of pages

 Calculate I/O cost of the join

 currentCost ← currentCost + joinCost

 Estimate size of intermediate result

 intermediateSize ← estimated result size

 END FOR

 IF currentCost < minimumCost THEN

 minimumCost ← currentCost

```
    bestPlan ← JoinOrder  
END IF
```

```
END FOR
```

```
RETURN bestPlan, minimumCost
```

```
END
```

Cost consideration

The optimizer considers:

- Number of tuples
- Number of pages
- Join order
- Intermediate result size
- Estimated I/O cost

The join ordering with the **minimum estimated I/O cost** is selected.

Q3. Pseudocode for Nested Loop Join and time complexity

Input: Relations R and S

Output: Joined relation

ALGORITHM Nested_Loop_Join(R, S)

```
BEGIN
```

```
    Result ← Empty
```

```
    FOR each tuple r in R DO
```

```
        FOR each tuple s in S DO
```

```
            IF JoinCondition(r, s) = TRUE THEN
```

```
                Add (r, s) to Result
```

```
            END IF
```

```
        END FOR
```

```
    END FOR
```

RETURN Result

END

Page-based version

Let:

- $B(R)$ = number of pages in R
- $B(S)$ = number of pages in S

The basic page nested-loop join cost is approximately:

$$B(R) + B(R) \times B(S)$$

If R contains $T(R)$ tuples and S contains $T(S)$ tuples, tuple comparisons are approximately:

$$T(R) \times T(S)$$

Therefore, the algorithm can be expensive for large relations because every tuple/page of one relation may be compared with the other relation.

Q4. Pseudocode for deadlock detection using Wait-For Graph

Input: Transaction wait relationships

Output: Deadlock detected or not detected

ALGORITHM Detect_Deadlock(WFG)

BEGIN

 Create a node for every active transaction

 Create directed edge $T_i \rightarrow T_j$
 if T_i is waiting for a lock held by T_j

 visited $\leftarrow$ Empty Set
 recursionStack $\leftarrow$ Empty Set

 FOR each transaction T DO

 IF T is not in visited THEN

```
    IF Detect_Cycle(T, visited, recursionStack) = TRUE THEN
        Display "Deadlock Detected"
        RETURN TRUE
    END IF
```

```
END IF
```

```
END FOR
```

```
Display "No Deadlock"
RETURN FALSE
```

```
END
```

Cycle detection function

```
FUNCTION Detect_Cycle(T, visited, recursionStack)
```

```
BEGIN
```

```
    Add T to visited
    Add T to recursionStack
```

```
    FOR each transaction U connected from T DO
```

```
        IF U is not in visited THEN
```

```
            IF Detect_Cycle(U, visited, recursionStack) = TRUE THEN
                RETURN TRUE
            END IF
```

```
        ELSE IF U is in recursionStack THEN
            RETURN TRUE
        END IF
```

```
    END FOR
```

```
    Remove T from recursionStack
```

```
    RETURN FALSE
```

```
END
```

If the Wait-For Graph contains a **cycle**, a deadlock exists.

Q5. Pseudocode for Basic Two-Phase Locking

Input: Transaction operations

Output: Controlled transaction execution

ALGORITHM Basic_2PL(Transaction T)

BEGIN

 phase $\leftarrow$ GROWING

 FOR each operation O in T DO

 IF O requires a lock THEN

 IF phase = GROWING THEN

 IF Lock is available THEN

 Grant lock

 ELSE

 Put transaction into WAIT state

 END IF

 END IF

 END IF

 Execute O

 IF Transaction decides to release a lock THEN

 phase $\leftarrow$ SHRINKING

 Release lock

 END IF

END FOR

 Release remaining locks

END

Basic 2PL rules

Growing Phase:

- Acquire locks
- No lock release

Shrinking Phase:

- Release locks
- No new locks can be acquired

This ensures conflict-serializable schedules.

Q6. Pseudocode for timestamp-based concurrency control using Thomas' Write Rule

Input: Transaction timestamp and data item timestamp

Output: Accepted or rejected write

Let:

- $TS(T)$ = timestamp of transaction T
- $R_TS(X)$ = largest timestamp of a transaction that read X
- $W_TS(X)$ = largest timestamp of a transaction that wrote X

ALGORITHM Thomas_Write_Rule(T, X)

BEGIN

IF $TS(T) < R_TS(X)$ THEN

// Write is too old
Reject write
Rollback T

ELSE IF $TS(T) < W_TS(X)$ THEN

// Write is obsolete
Ignore the write

ELSE

Perform Write(X)
 $W_TS(X) \leftarrow TS(T)$

END IF

END

For a read operation

ALGORITHM Timestamp_Read(T, X)

BEGIN

IF $TS(T) < W_TS(X)$ THEN

 Reject read

 Rollback T

ELSE

 Perform Read(X)

$R_TS(X) \leftarrow \text{MAX}(R_TS(X), TS(T))$

END IF

END

Thomas' Write Rule allows an obsolete write to be ignored when it is already superseded by a newer write, instead of unnecessarily rolling back the transaction.

Q7. Pseudocode for ARIES Recovery

ARIES consists of three main phases:

1. Analysis
2. Redo
3. Undo

ALGORITHM ARIES_Recovery(Log)

BEGIN

 // Phase 1: Analysis

 Read the log from the latest checkpoint

 Identify active transactions

 Identify committed transactions

 Identify dirty pages

Build Transaction Table
Build Dirty Page Table

// Phase 2: Redo

Find the starting point for REDO

FOR each log record from REDO starting point DO

 IF operation needs REDO THEN
 Repeat the logged operation
 END IF

END FOR

// Phase 3: Undo

Identify loser transactions

FOR each loser transaction T DO

 Traverse its log records backward

 Undo each required operation

 Write compensation log record

END FOR

Write END records for completed transactions

END

Result

After Analysis, Redo and Undo phases, the database is restored to a consistent state.

Q8. Pseudocode for Shadow Paging Recovery

Input: Database pages and transaction updates

Output: Recovered database

ALGORITHM Shadow_Paging_Recovery()

BEGIN

 Create Shadow_Page_Table

 Current_Page_Table $\leftarrow$ Copy(Shadow_Page_Table)

 WHILE Transaction is executing DO

 IF Transaction modifies page P THEN

 Create a new copy of page P

 Update new page with modified data

 Change Current_Page_Table
 to point to the new page

 END IF

 END WHILE

 IF Transaction commits successfully THEN

 Shadow_Page_Table $\leftarrow$ Current_Page_Table

 Commit changes

 ELSE IF System failure occurs THEN

 Current_Page_Table $\leftarrow$ Shadow_Page_Table

 Discard uncommitted page changes

 END IF

END

Main idea

The **Shadow Page Table** always points to the last stable database version. Therefore, after a failure, the system can restore the shadow page table without performing traditional UNDO operations.

Q9. Pseudocode for Immediate Update recovery using UNDO/REDO

Input: Transaction log

Output: Recovered database

ALGORITHM Immediate_Update_Recovery(Log)

BEGIN

 Read log records after the latest checkpoint

 Identify committed transactions

 Identify uncommitted transactions

 // REDO phase

 FOR each committed transaction T DO

 FOR each log record of T in forward order DO

 REDO the operation

 END FOR

 END FOR

 // UNDO phase

 FOR each uncommitted transaction T DO

 FOR each log record of T in reverse order DO

 UNDO the operation

 END FOR

 Write ABORT record

END FOR

Write COMMIT/END records for completed transactions

END

Recovery rule

Committed Transaction → REDO

Uncommitted Transaction → UNDO

Immediate Update may require both UNDO and REDO because changes can be written to the database before the transaction commits.

Q10. Pseudocode for a database connectivity manager

Input: Database connection request and SQL query

Output: Query result or rollback

ALGORITHM Database_Connectivity_Manager()

BEGIN

Initialize Connection Pool

WHILE Application is running DO

TRY

Get available connection from pool

IF connection is not available THEN

Wait until connection becomes available

END IF

Begin Transaction

Receive SQL Query

Prepare SQL Statement

Bind required parameters

Execute SQL Query

IF Query is successful THEN

Commit Transaction

Return Result

ELSE

Rollback Transaction

Return Error

END IF

Return connection to pool

CATCH DatabaseException

Rollback Transaction

Log exception details

Return connection to pool

Display "Database operation failed"

END TRY

END WHILE

Close all connections in the pool

END

Functions of the connectivity manager

1. **Connection Pooling:** Reuses database connections instead of creating a new connection for every query.
2. **Query Execution:** Sends SQL statements to the database and processes the result.
3. **Transaction Management:** Commits successful operations.
4. **Exception Handling:** Detects database errors.
5. **Rollback:** Restores the previous consistent state when an operation fails.
6. **Connection Release:** Returns the connection to the pool after completion.

Conclusion

This design improves performance through connection pooling and maintains database consistency using transaction commit and rollback mechanisms.